CS 0007: Introduction to Java Lecture 20

Nathan Ong
University of Pittsburgh
November 22, 2016

Long Story Short

- Scanner has a buffering quirk. When scanning for a token (an item like `boolean`, `int`) that is not a line, the scanner reads up to the new line character, but does not consume it.
- When you call `nextLine()`, it sees the new line, assumes it is done reading, and returns an empty `String`.

A Reminder

- The act of receiving input is called reading
- Relevant classes that do reading are Readers

A Re-examination

- **`System.in`**
- An **`InputStream`**
- If we examine the API, **`InputStream`**s can only read bytes. Not very helpful.
- What could we try?

`InputStreamReader`

- Now we can read characters, but having to combine all the characters is difficult
- Why don’t we use the suggestion from the API?

`BufferedReader`

- This thankfully allows us to read full lines of input
- The input needs to be processed depending on your needs
- Let us first learn how to use **`BufferedReader`** correctly

Exceptions

- An exception is an event meant to disrupt the flow of execution
- To throw an exception is to see an exception being raised
- To catch an exception is to acknowledge a thrown exception
- You probably have already encountered some while debugging your code
- We will focus on two of three types, which are more common

Exception Types

- Checked
  - Exceptions that need to have a contingency plan should the exception arise
  - These stop the compiler from running
  - Example: **`IOException`**
- Unchecked
  - Exceptions that tend to indicate the program has a flaw during its execution
  - These stop the program in the middle of execution
  - Example: **`ArrayIndexOutOfBoundsException`**

Dealing with Exceptions

- Checked
  - `try`, `catch`, `finally`
- Unchecked
  - Fix your program code

Keywords in Context

- Everything inside the `try` block indicates the section of code that may throw exceptions
- Inside the `catch`'s parentheses is the expected exception to process, and the name given to it (usually '**e**')
- Inside the `catch` block indicates the section of code that should be executed if the exception is caught

Some FAQ

- How do I know when an exception is thrown?
  - Check the API for relevant methods
- How do I know which exception is thrown?
  - Check the API
- How much code should I surround with `try`?
  - Only as much as you need
  - Note that scope applies here as well
- Can I catch multiple exceptions?
  - Yes, you can use several `catch` blocks
  - We'll see an example soon

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class BRExample
{
 public static void main(String[] args)
 {
 String line = "";
 System.out.println("Type something!");
 try
 {
 BufferedReader reader = new
     BufferedReader(new
     InputStreamReader(System.in));
 line = reader.readLine();
 reader.close();
 }
 catch (IOException e)
 {
 e.printStackTrace();
 }
 System.out.println(line + " is what you wrote.");
 }//end method main
}//End class BRExample
```

Converting Strings into Useful Primitives

- **`Scanner`** gave us useful methods like **`nextInt()`** or **`nextDouble()`**. We can do the same with **`BufferedReader`**, but not directly.
- We need to take another trip to the API, specifically for the classes that back the primitives.
- Let’s look at **`Integer`**

Example

```
public class Converter
{
 public static int convertToInt(String line)
 {
 try
 {
 return Integer.parseInt(line);
 }
 catch (NumberFormatException e)
 {
 //??
 }
 }//end method main
}//End class Converter
```

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class BRExample2
{
 public static void main(String[] args)
 {
 int inputNum = -1;
 System.out.println("Type an integer!");
 try
 {
 BufferedReader reader = new
     BufferedReader(new
     InputStreamReader(System.in));
 String line = reader.readLine();
 inputNum = Converter.convertToInt(line);
 reader.close();
 }
 catch (IOException e)
 {
 e.printStackTrace();
 }
 System.out.println(inputNum + " + 1 = " + (inputNum+1));
 }//end method main
}//End class BRExample2
```

A Note about `Integer`

- **`Integer`** is a wrapper class around the primitive **`int`**
- All primitives have wrapper classes
- Consult the API

File Input/Output

- Files are the biggest way we interact with computer data.
- Your Word documents, your game save files, your Java source code, all of them are stored as files.
- We want to read them too!
- Let’s check **`BufferedReader`**’s API for any clues.

File Input/Output

- Files are the biggest way we interact with computer data.
- Your Word documents, your game save files, your Java source code, all of them are stored as files.
- We want to read them too!
- Let’s check **`BufferedReader`**’s API for any clues.

```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.io.FileNotFoundException;

public class BRWithFiles
{
 public static void main(String[] args)
 {
 int inputNum = -1;
 System.out.println("Type an integer!");
 try
 {
 BufferedReader reader = new BufferedReader(new
                FileReader("test.txt"));
 String line = reader.readLine();
 inputNum = Converter.convertToInt(line);
 reader.close();
 }
 catch (FileNotFoundException e)
 {
 System.err.println("The file was not found");
 }
 catch (IOException e)
 {
 e.printStackTrace();
 }
 System.out.println("The number in the file was: " + inputNum);
 }//end method main
}//End class BRWithFiles
```

Writers and Writing

- The act of providing output is called writing
- Relevant classes that do writing are Writers
- How do I write to a File?

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class InputAndOutput
{
 …
 public static String getInput(String prompt)
 {
 System.out.println(prompt);
 String input = null;
 try
 {
 BufferedReader reader = new BufferedReader(new
     InputStreamReader(System.in));
 input = reader.readLine();
 reader.close();
 }
 catch (IOException e)
 {
 e.printStackTrace();
 }
 return input;
 }//end method getInput(String)
}//End class InputAndOutput
```

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class InputAndOutput
{
     …
     public static void writeOutput(String lines, String filename)
     {
     try
     {
     FileWriter writer = new FileWriter(filename);
     writer.write(lines);
     writer.flush();
     writer.close();
     }
     catch (IOException e)
     {
     e.printStackTrace();
     }
     }//end method writeOutput(String,String)
}//End class InputAndOutput
```

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class InputAndOutput
{
     …
     public static void main(String[] args)
     {
     String filename = getInput("What
     file would you like to
     create?");
     String toStore = getInput("What
     would you like to store in
     the file?");
     writeOutput(filename,toStore);
     }//end method main
}//End class InputAndOutput
```